



UAI

Universidad Abierta Interamericana

FACULTAD DE TECNOLOGÍA INFORMÁTICA

INGENIERIA DE SOFTWARE

Alumno: Oven Pereyra

Localización: Castelar

Docente: Julián Rodríguez Escobedo

Turno: Noche

Año de cursada: 2026

Comisión: 3A

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 2/6/2026
	Alumno: Oten Pereyra			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	2 Entrega				

Contenido	
T01. Arquitectura Base	3
Objetivo	3
Descripción detallada	3
Diagrama de clases	5
T02. Gestión de Log In / Log Out	7
Objetivo	7
Descripción detallada	8
T03. Gestión de Encriptado	12
Objetivo	12
Descripción detallada	12
T04. Gestión de Perfiles de Usuarios	17
a. Objetivo	17
b. Descripción detallada	17
T05. Gestión de múltiples idiomas	22
a. Objetivo	22
Diagrama de clase	24
.....	24
Diagrama de caso de uso	24
Diagrama de secuencia	27
.....	27
DER	28
T06. Gestión de Bitácora y Control de cambios	28

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oten Pereyra		Legajo:	
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			Versión
2 Entrega				

T01. Arquitectura Base

Objetivo

Definir, documentar e implementar la arquitectura base del sistema de información de la Agencia de Quiniela, estableciendo una estructura de cuatro capas que garantice la separación de responsabilidades, la cohesión, el bajo acoplamiento y la reutilización de componentes. Esta arquitectura servirá como fundamento para el desarrollo de todas las funcionalidades del sistema.

Descripción detallada

El sistema se organiza en cuatro capas bien diferenciadas, cada una con una responsabilidad específica:

Capa UI (User Interface)

Es la capa de presentación del sistema. Contiene los formularios Windows Forms que interactúan directamente con el usuario. Esta capa solo se comunica con la capa BLL, nunca directamente con DAL ni con Mapper.

- Form1: formulario principal de inicio de sesión con campos de usuario y contraseña.
- FormABM: formulario para la gestión (alta, baja, modificación) de usuarios, accesible solo para el rol admin. Muestra la grilla de usuarios y controles de edición.
- FormComposite: formulario para editar el árbol jerárquico de permisos de un usuario específico, con visualización en TreeView y listas de familias y parientes.

Capa BLL (Business Logic Layer)

Contiene toda la lógica y las reglas de negocio del sistema. Es la capa intermediaria entre la UI y el acceso a datos. Aquí se realizan validaciones, se aplican patrones de diseño (Singleton y Observer) y se invoca el encriptado de contraseñas antes de persistirlas.

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oven Pereyra			Legajo:
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			Versión
2 Entrega				

- **UsuarioService:** gestiona el login, logout y el ABM de usuarios. También administra el árbol de permisos Composite en memoria, con métodos para agregar familias, agregar parientes, enlazarlos y guardarlos.
- **SesionManager:** implementa el patrón Singleton para mantener el estado de la sesión activa. Permite consultar desde cualquier capa si hay un usuario logueado y cuál es su rol.
- **Bitacora:** implementa el patrón Singleton de logging. Crea un archivo .log nuevo por cada ejecución y registra cada evento relevante (login, logout, ABM, cambio de permisos, cambio de idioma, errores de BD).
- **GestorIdioma:** Subject del patrón Observer. Singleton que mantiene el idioma actual y notifica a todos los formularios suscritos cuando el idioma cambia.
- **PermisoSerializer:** clase estática que serializa y deserializa el árbol Composite a un formato de texto plano lineal (G|nombre|cantHijos para grupos, L|nombre para hojas), utilizado para persistir los permisos en la base de datos.

Capa DAL (Data Access Layer)

Es responsable exclusivamente del acceso a la base de datos. Contiene las consultas SQL parametrizadas y las operaciones de lectura y escritura. Nunca contiene lógica de negocio.

- **UsuarioDAL:** implementa los métodos Login, ObtenerTodos, ObtenerPorId, Agregar, Modificar, Eliminar, ExisteUsuario y ActualizarPermisos sobre la tabla Usuarios. También expone el método estático HashSHA256 y el método InicializarBaseDatos que crea la BD, la tabla y el usuario admin si no existen.

Capa Mapper (Modelos)

Contiene las clases que representan las entidades del dominio y las interfaces compartidas. Son objetos de transferencia de datos (DTO) e interfaces utilizados por todas las capas. No contienen lógica de negocio. Al no depender de ninguna otra capa, evita dependencias circulares.

- **Usuario:** entidad con propiedades Id, NombreUsuario, Clave (hash SHA-256), Rol, Permisos (árbol serializado) y TipoPermiso.
- **GrupoPermiso:** nodo compuesto del patrón Composite. Implementa IComponentePermiso y contiene una lista de hijos (otros grupos u hojas).
- **PermisoLeaf:** nodo hoja del patrón Composite. Representa un permiso atómico individual.
- **IComponentePermiso:** interfaz base del Composite, con propiedad Nombre y método ObtenerInfo().
- **Resultado:** DTO de respuesta para operaciones de negocio, con propiedades Ok (bool) y Mensaje (string).
- **IObservadorIdioma:** interfaz Observer que deben implementar todos los formularios que reaccionen al cambio de idioma. Expone el método ActualizarIdioma().

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oven Pereyra			Fecha 2/6/2026
	Localización:	Comisión: 3A	Turno:	Legajo:
	Castelar		Noche	Año: 2026
	<Nombre del Sistema>			Versión
2 Entrega				

Esquema de persistencia

La capa DAL utiliza ADO.NET con SqlConnection y SqlCommand parametrizados para todas las operaciones sobre la base de datos SQL Server. Para consultas se utiliza SqlDataReader, y para altas, bajas y modificaciones se utiliza ExecuteNonQuery. Todas las conexiones se abren y cierran dentro de bloques using para garantizar la liberación de recursos.

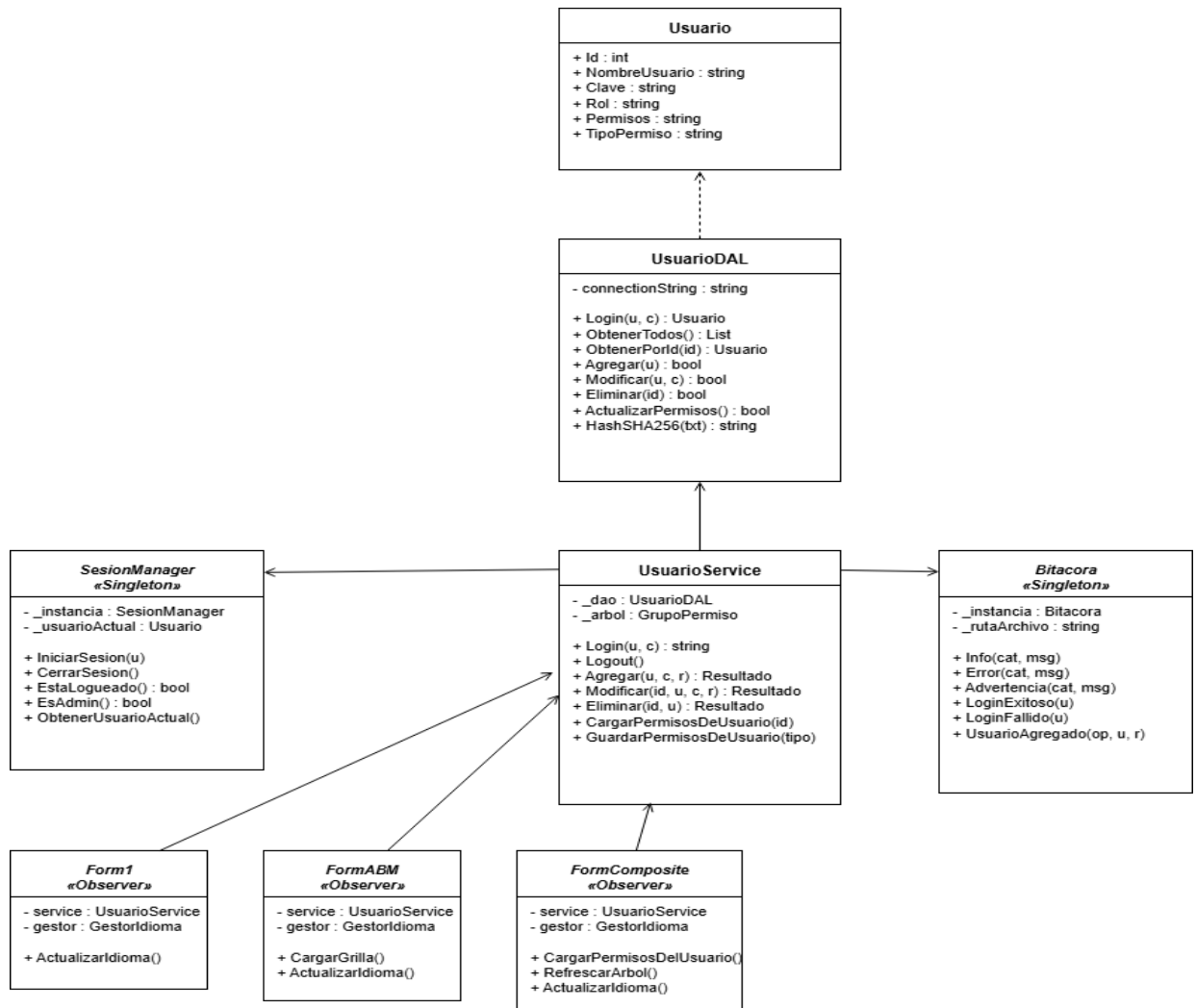
Mapa de navegacion

El flujo de navegación del sistema es el siguiente:

- Al iniciar la aplicación, Program.cs inicializa la Bitácora y llama a InicializarBaseDatos(). Si falla, muestra error y cierra.
- Se presenta Form1 con los campos de login.
- Tras un login exitoso, Form1 oculta los controles de acceso y muestra los botones según el rol.
- Si el usuario es admin, puede abrir FormABM para gestionar usuarios, y desde allí abrir FormComposite para editar los permisos de un usuario.
- El botón de cambio de idioma (esquina superior derecha) notifica a todos los formularios abiertos vía GestorIdioma.
- Al cerrar sesión, Form1 restaura los controles de login.

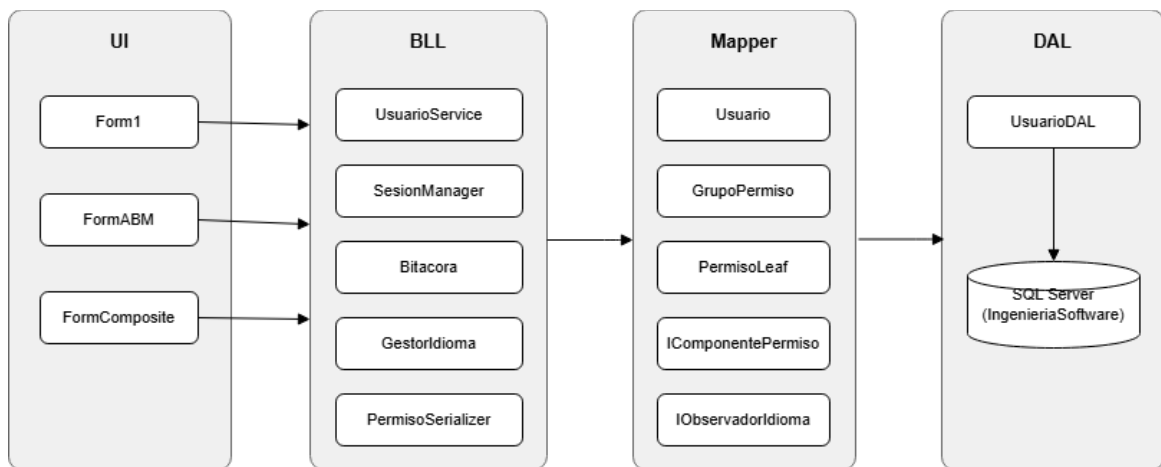
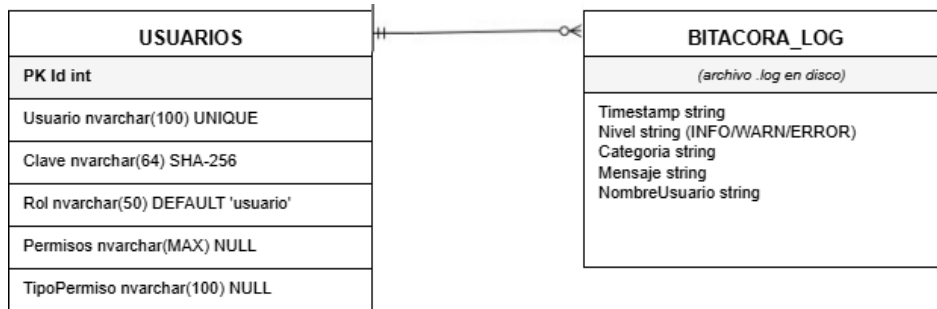
Diagrama de clases

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
 UAI Universidad Abierta Interamericana	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 2/6/2026
	Alumno: Oven Pereyra			Legajo:	
	Localización:	Comisión: 3A	Turno:	Año: 2026	
	Castelar		Noche		Versión
	<Nombre del Sistema>				
2 Entrega					



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 2/6/2026
	Alumno: Oven Pereyra			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	2 Entrega				

DER



T02. Gestion de Log In / Log Out

Objetivo

Implementar el mecanismo de autenticación y cierre de sesión del sistema, permitiendo verificar la identidad del usuario mediante su nombre de usuario y

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oten Pereyra			Fecha 2/6/2026
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026
	<Nombre del Sistema> 2 Entrega			Versión

contraseña. El sistema gestiona el estado de la sesión mediante el patrón de diseño Singleton (SesionManager), garantizando que solo exista una instancia activa de sesión en todo momento, y registra cada evento en la Bitácora.

Descripcion detallada

Proceso de Log In

1. El usuario ingresa su nombre de usuario y contraseña en Form1.
2. La UI invoca el método Login(nombreUsuario, clave) de UsuarioService.
3. UsuarioService delega en UsuarioDAL.Login(), que hashea la clave con SHA-256 antes de compararla con la BD.
4. Si las credenciales coinciden, UsuarioDAL retorna el objeto Usuario; UsuarioService lo pasa a SesionManager.Instancia.IniciarSesion(usuario).
5. Form1 oculta los controles de login y muestra los botones según el rol.
6. Si no coinciden, se muestra un mensaje de error y se registra el intento fallido en la Bitácora.

Proceso de Log Out

1. El usuario hace clic en el botón Cerrar sesión de Form1.
2. La UI invoca el método Logout() de UsuarioService.
3. UsuarioService llama a SesionManager.Instancia.CerrarSesion(), que establece usuarioActual en null.
4. Form1 restaura los campos de login y oculta los botones de administración.

Patron Singleton en SesionManager

SesionManager implementa el patrón Singleton para garantizar que solo exista una instancia del gestor de sesión durante toda la ejecución del sistema. El constructor es privado y la instancia se crea de manera lazy (solo cuando se accede por primera vez). Esto asegura que el usuario autenticado sea accesible desde cualquier capa de la aplicación sin necesidad de pasar parámetros entre formularios.

Patrón Singleton en SesionManager

SesionManager implementa el patrón Singleton para garantizar que solo exista una instancia del gestor de sesión durante toda la ejecución del sistema. El constructor es privado y la instancia se crea de manera lazy (solo cuando se accede por primera

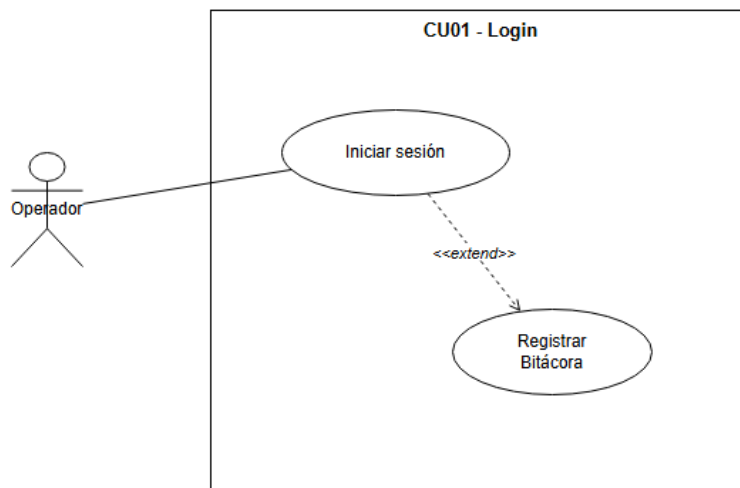
UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oven Pereyra		Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026
	<Nombre del Sistema> 2 Entrega			Versión

vez). Esto asegura que el usuario autenticado sea accesible desde cualquier capa de la aplicación sin necesidad de pasar parámetros entre formularios.

Bitácora de sesión

Cada evento de login y logout queda registrado en el archivo de log de la ejecución actual con la hora, el nombre de usuario y el tipo de evento (LoginExitoso, LoginFallido, Logout). Este registro permite auditar el acceso al sistema.

Diagrama de caso de uso Login



ID Y NOMBRE:	CU01- Iniciar sesion
ESTADO:	Aprobado
ACTOR PRINCIPAL: Operador	
ACTORES SECUNDARIO: -	
PRECONDICIONES: El usuario debe estar registrado en el sistema	
PUNTOS DE EXTENSIÓN: Registrar auditoria	
CONDICIÓN: El usuario ingresa usuario y contraseña	
ESCENARIO PRINCIPAL: 1. El operador ingresa usuario y contraseña	

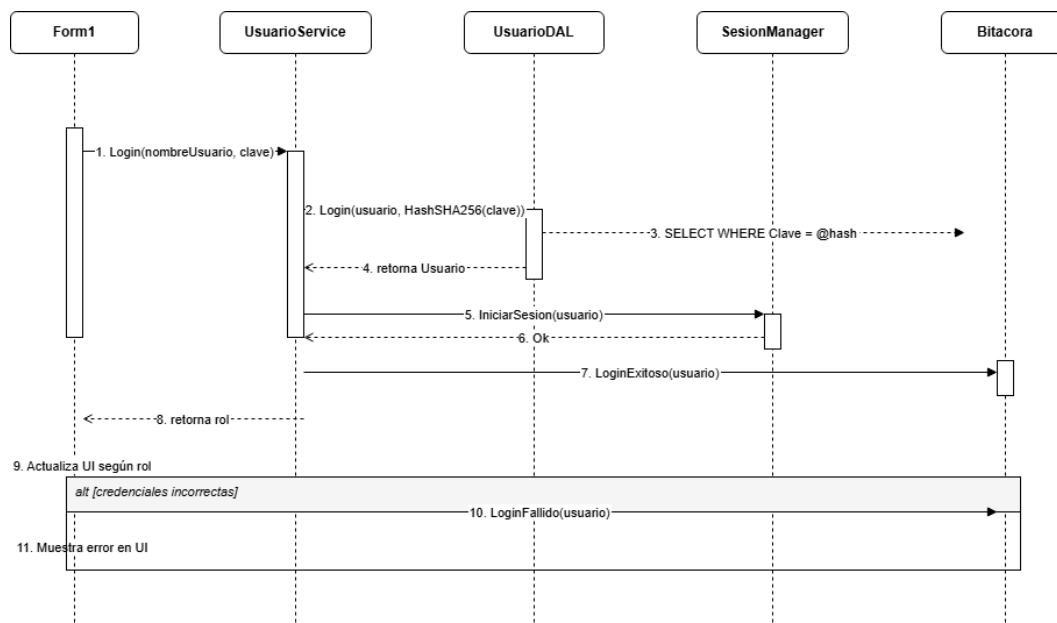
UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oven Pereyra		Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026
	<Nombre del Sistema> 2 Entrega			Versión

2. El sistema encripta la contraseña (hash SHA-256)
3. El sistema busca el usuario en la base de datos
4. El sistema compara las credenciales
5. Si son correctas, inicia sesión
6. Se registra la auditoria
7. Se muestra la pantalla principal

FLUJOS ALTERNATIVO: 4a Credenciales incorrectas- se muestra error y no inicia sesión

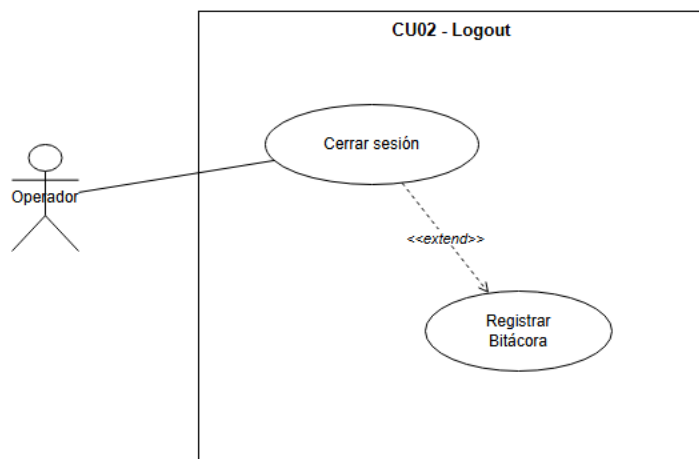
POSTCONDICIONES: Usuario autenticado en el sistema.

Diagrama de secuencia Login



UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oven Pereyra		Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026
	<Nombre del Sistema> 2 Entrega			Versión

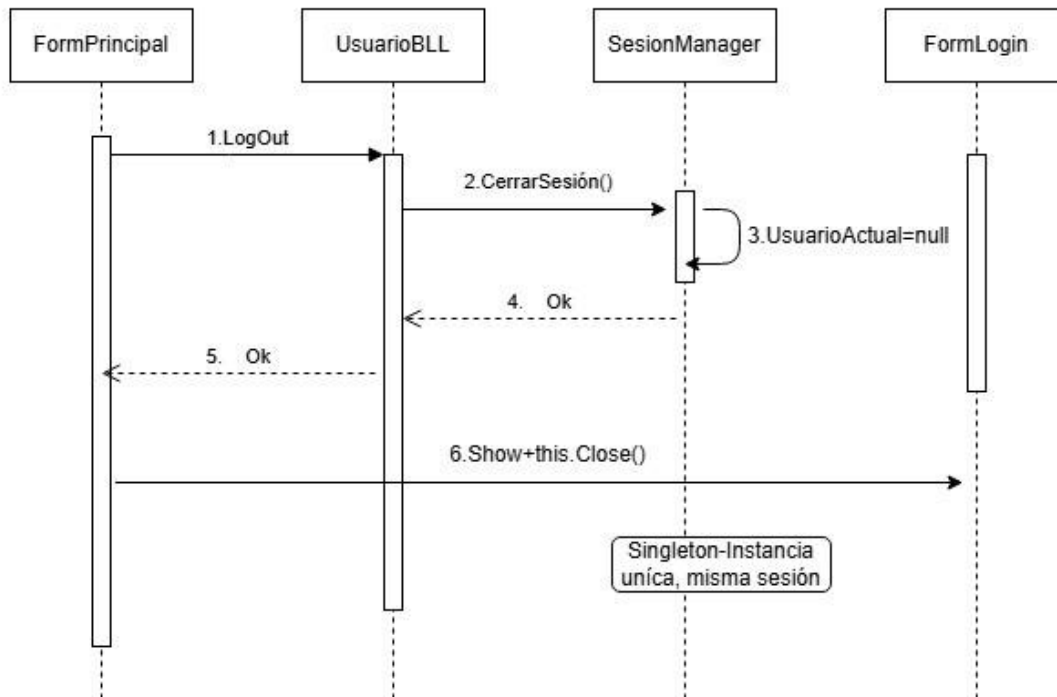
Diagrama de caso de uso LogOut



ID Y NOMBRE:	CU02- Cerrar sesion
ESTADO:	Aprobado
ACTOR PRINCIPAL: Operador	
ACTORES SECUNDARIO: -	
PRECONDICIONES: El usuario debe tener una sesión activa	
PUNTOS DE EXTENSIÓN: Registrar auditoria	
CONDICIÓN: El usuario selecciona cerrar sesión.	
ESCENARIO PRINCIPAL: 1. El operador solicita cerrar sesión 2. El sistema elimina la sesion actual 3. El sistema limpia el usuario activo 4. El sistema compara las credenciales 5. Se registra la auditoria 6. Se redirige al Login	
FLUJOS ALTERNATIVO: -	
POSTCONDICIONES: Sesión finalizada	

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oven Pereyra		Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026
	<Nombre del Sistema> 2 Entrega			Versión

Diagrama de secuencia LogOut



T03. Gestion de Encriptado

Objetivo

Proteger los datos sensibles del sistema, en particular las contraseñas de los usuarios, mediante la aplicación de un algoritmo de hash criptográfico. El sistema utiliza SHA-256 como mecanismo de hash unidireccional, garantizando que en ningún momento se almacene ni se transmita la contraseña en texto plano.

Descripcion detallada

Algoritmo utilizado: SHA-256

SHA-256 (Secure Hash Algorithm 256 bits) es un algoritmo de hash criptográfico unidireccional. Toma un texto de entrada de cualquier longitud y produce un hash de longitud fija (64 caracteres hexadecimales). Sus propiedades fundamentales son:

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oten Pereyra			Legajo:
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			
2 Entrega				
Fecha				
2/6/2026				
Versión				

- Determinismo: el mismo input siempre produce el mismo hash.
- Irreversibilidad: es computacionalmente imposible obtener el texto original a partir del hash.
- Efecto avalancha: un cambio mínimo en el input produce un hash completamente diferente.
- Unicidad: es extremadamente improbable que dos inputs distintos produzcan el mismo hash (colisión).

Implementacion en el sistema

El método estático HashSHA256(string texto) se encuentra en la clase UsuarioDAL (capa DAL). Utiliza la clase SHA256 del namespace System.Security.Cryptography de .NET Framework. El método convierte el texto a bytes UTF-8, aplica el algoritmo y retorna el hash como cadena hexadecimal en minúsculas de 64 caracteres.

Uso en el flujo del sistema

- Al crear un usuario: la contraseña ingresada es hasheada antes de ser persistida en la base de datos.
- Al modificar un usuario: si se proporciona una nueva contraseña, es hasheada antes de actualizar el registro. Si el campo contraseña queda vacío, se mantiene el hash actual.
- Al iniciar sesión: la contraseña ingresada es hasheada en el momento de la consulta SQL (WHERE Clave = @hash). Nunca se descripta.
- Al inicializar la BD: el usuario admin se crea con la contraseña 'admin123' ya hasheada. En cada arranque se actualiza el hash del admin para garantizar consistencia.

Ejemplo de aplicacion

Contraseña ingresada: admin123

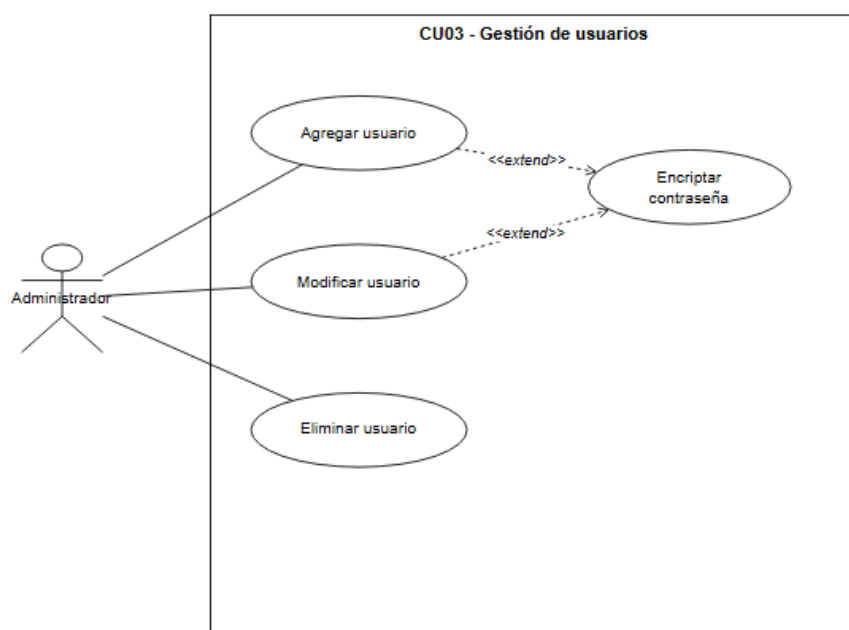
Hash SHA-256 resultante:

240be518fabd2724ddb6f04eeb1da5967448d7e8f9324d292ec231629a657db6

Este valor es el que se almacena en la columna Clave de la tabla Usuarios.

Diagrama de caso de uso

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oven Pereyra		Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026
	<Nombre del Sistema> 2 Entrega			Versión



ID Y NOMBRE:	CU03- Gestion de usuarios
ESTADO:	Aprobado
ACTOR PRINCIPAL: Administrador	
ACTORES SECUNDARIO: -	
PRECONDICIONES: El administrador debe estar autenticado	
PUNTOS DE EXTENSIÓN: Encriptar contraseña	
CONDICIÓN: El administrador accede al módulo de gestión de usuarios	
ESCENARIO PRINCIPAL: 1. El administrador selecciona la opcion crear usuario 2. Ingresa los datos del usuario 3. El sistema encripta la contraseña 4. El sistema guarda el usuario en la base de datos 5. Se confirma la operación	

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oten Pereyra			Fecha 2/6/2026
	Localización: Castelar	Comisión: 3A	Turno: Noche	Legajo: Año: 2026
	<Nombre del Sistema> 2 Entrega			Versión

FLUJOS ALTERNATIVO: A1-Modificar usuario

1. El administrador selecciona un usuario existente
2. Modifica los datos
3. El sistema encripta la contraseña (si corresponde)
4. El sistema actualiza el usuario
5. Se confirma la operación

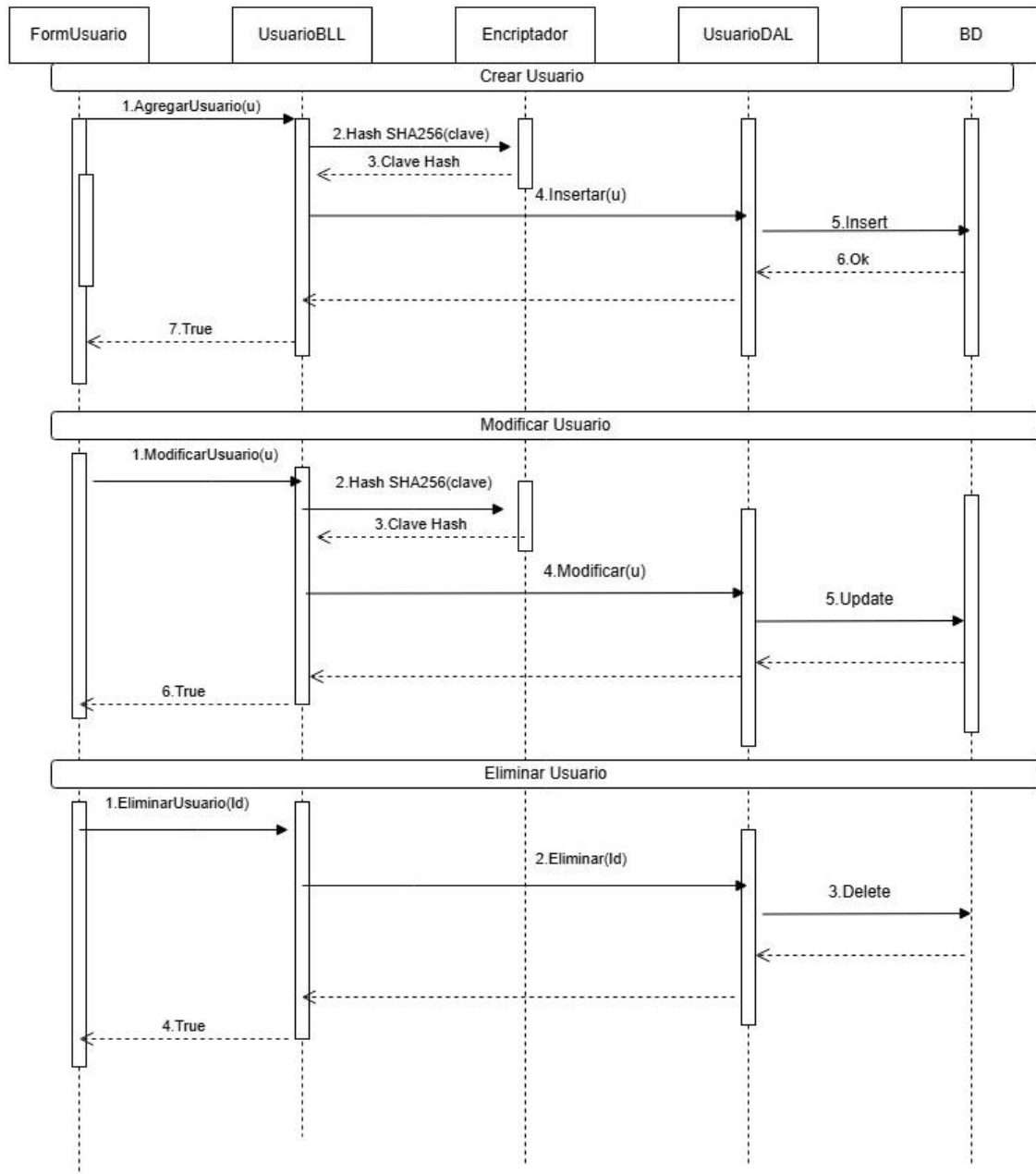
A2-Eliminar usuario

1. El administrador selecciona un usuario
2. El sistema elimina el usuario
3. Se confirma la operación

POSTCONDICIONES: Los cambios en los usuarios quedan registrados en el sistema.

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oten Pereyra		Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026
	<Nombre del Sistema> 2 Entrega			Versión

Diagrama de secuencia



UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oven Pereyra		Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026
	<Nombre del Sistema> 2 Entrega			Versión

T04. Gestion de Perfiles de Usuarios

a. Objetivo

Implementar la gestión de permisos del sistema mediante el patrón de diseño Composite, permitiendo definir permisos atómicos (hojas, representados por PermisoLeaf) y permisos compuestos (grupos, representados por GrupoPermiso), organizarlos en una jerarquía en árbol y asignarlos a cada usuario de manera individual. El árbol se visualiza mediante un TreeView interactivo en FormComposite y se persiste en la base de datos en formato de texto serializado.

b. Descripción detallada

Patrón de diseño Composite

El patrón Composite permite tratar objetos individuales y composiciones de objetos de manera uniforme a través de una interfaz común. En este sistema se aplica de la siguiente forma:

- IComponentePermiso (interfaz): define la propiedad Nombre y el método ObtenerInfo(). Es implementada tanto por GrupoPermiso como por PermisoLeaf, lo que permite tratarlos polimórficamente en el árbol.
- GrupoPermiso (nodo compuesto): agrupa otros GrupoPermiso o PermisoLeaf en una lista interna de hijos. Expone métodos Agregar(), Quitar() y Hijos(). Representa categorías como 'Administrador', 'gestionUser', 'gestionProducto'.
- PermisoLeaf (hoja): representa un permiso atómico e indivisible (ej: 'addUser', 'deleteProducto'). No tiene hijos.

Árbol por defecto

Al crear un usuario nuevo, UsuarioService construye el siguiente árbol predeterminado:

- Raiz
 - └─ Administrador (GrupoPermiso)
 - └─ gestionUser (GrupoPermiso)
 - └─ addUser (PermisoLeaf)
 - └─ updateUser (PermisoLeaf)
 - └─ deleteUser (PermisoLeaf)
 - └─ gestionProducto (GrupoPermiso)
 - └─ addProducto (PermisoLeaf)
 - └─ updateProducto (PermisoLeaf)
 - └─ deleteProducto (PermisoLeaf)

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oven Pereyra		Legajo:	
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			Versión
2 Entrega				

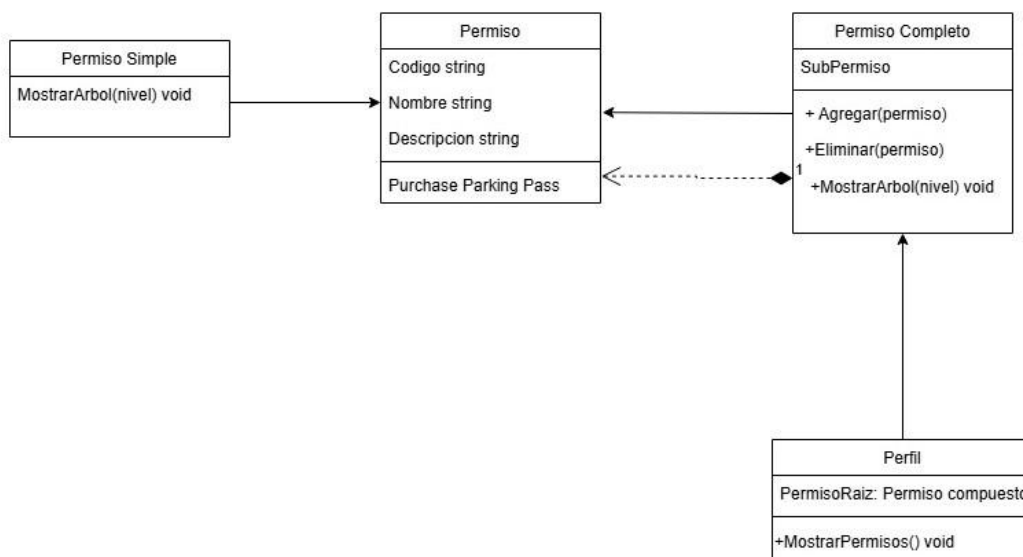
Serialización del árbol

PermisoSerializer convierte el árbol a texto plano para persistirlo en la columna Permisos de la tabla Usuarios. Formato: cada nodo ocupa una línea, prefijada con 'G|nombre|cantHijos' para grupos y 'L|nombre' para hojas. La deserialización reconstruye el árbol leyendo las líneas en preorden. Los nombres no pueden contener el carácter '|'.

Interfaz visual — FormComposite

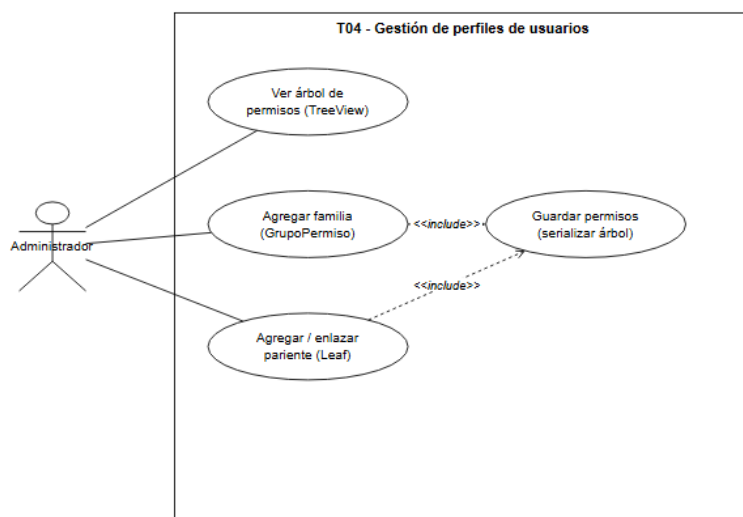
FormComposite muestra el árbol actual en un TreeView (izquierda) y expone controles para agregar familias (grupos), agregar parientes (hojas libres), enlazarlos a familias, eliminar nodos y guardar. Un segundo TreeView de preview muestra en tiempo real el resultado de la operación antes de confirmarla. Al guardar, el árbol se serializa y se persiste via UsuarioService.GuardarPermisosDeUsuario().

Diagrama de clases



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 2/6/2026
	Alumno: Oven Pereyra			Legajo:	
	Localización:	Comisión: 3A	Turno:	Año: 2026	
	Castelar		Noche		Versión
	<Nombre del Sistema>				
2 Entrega					

Diagrama de caso de uso



ID Y NOMBRE:	CU04- Gestión de perfiles de usuarios
ESTADO:	Aprobado
ACTOR PRINCIPAL: Administrador	
ACTORES SECUNDARIO: -	
PRECONDICIONES: El actor debe haber iniciado con un usuario.	
PUNTOS DE EXTENSIÓN: N/A	
CONDICIÓN: -	

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oven Pereyra			Fecha 2/6/2026
	Localización: Castelar	Comisión: 3A	Turno: Noche	Legajo: Año: 2026
	<Nombre del Sistema> 2 Entrega			Versión

ESCENARIO PRINCIPAL:

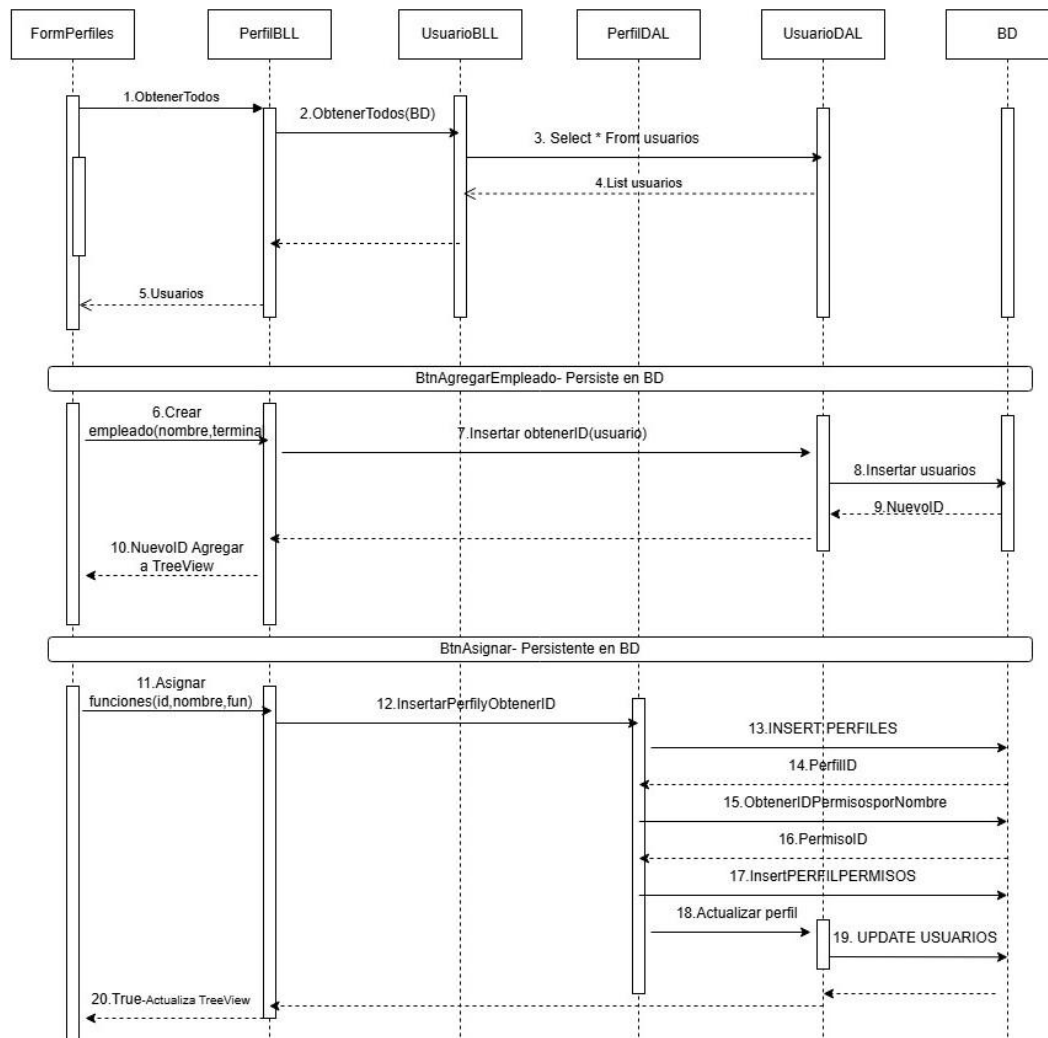
1. El Administrador ingresa en la pantalla de “gestión de perfiles”
2. El sistema consulta en la base de datos y dibuja la jerarquía actual de los usuarios y sus permisos
3. El usuario ingresa el nombre de un nuevo empleado y presiona “agregar”
4. El sistema crea el registro le genera un nro de terminal y actualiza el árbol visual en tiempo real
5. El usuario marca una o más casillas de la lista de “funciones disponibles”
6. El Administrador presiona “asignar funciones al empleado”
7. El sistema guarda la relación en la base de datos y redibuja los nodos del árbol para mostrar los nuevos permisos

FLUJOS ALTERNATIVO: 3a. Nombre en blanco: si al intentar “ agregar empleado” el campo está vacío, el sistema aborta la operación.

POSTCONDICIONES: El empleado queda registrado en la base de datos con una lista de permisos guardada.

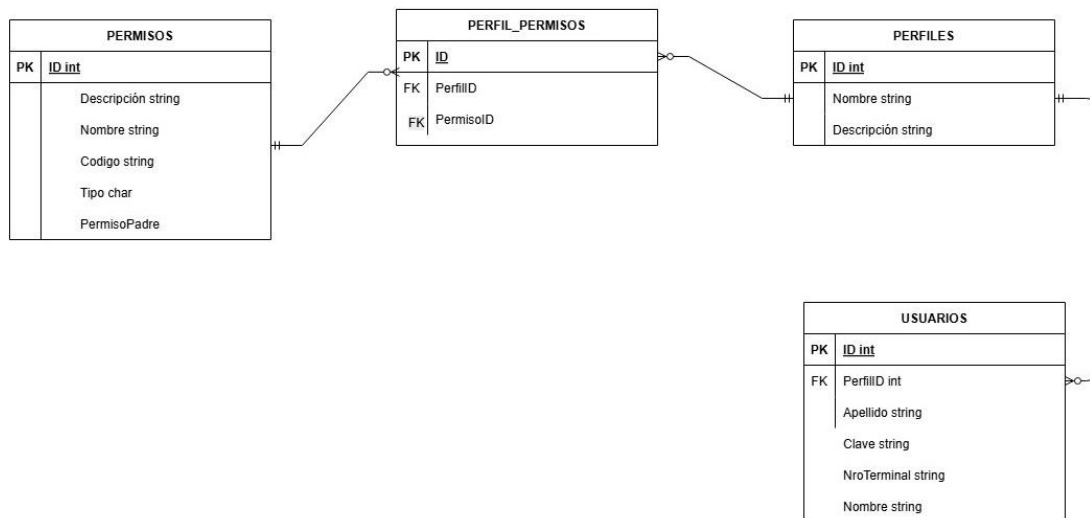
Diagrama de secuencia

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oven Pereyra		Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026
	<Nombre del Sistema> 2 Entrega			Versión



UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oven Pereyra		Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026
	<Nombre del Sistema> 2 Entrega			Versión

DER



T05. Gestión de múltiples idiomas

a.Objetivo

Proveer a la aplicación de un mecanismo dinámico y en tiempo real que permita modificar el idioma de la interfaz gráfica sin necesidad de reiniciar el sistema. El sistema soporta Español e Inglés y garantiza que todas las leyendas, títulos y mensajes visibles para el usuario sean traducidos automáticamente en todos los formularios abiertos simultáneamente, utilizando el patrón de diseño Observer.

b.Descripción detallada

GestorIdioma actúa como Subject (publicador). Implementa el patrón Singleton y mantiene el idioma actual (Español o Inglés) y una lista de formularios suscritos

UNIVERSIDAD ABIERTA INTERAMERICANA				
Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oten Pereyra		Legajo:	
	Localización:	Comisión: 3A	Turno:	Año: 2026
	Castelar		Noche	
	<Nombre del Sistema>			Versión
2 Entrega				

(observers). Cuando el idioma cambia, notifica a todos los observers llamando al método ActualizarIdioma() de cada uno.

- GestorIdioma (Subject/Singleton): expone los métodos Suscribirse(), Desuscribirse() y CambiarIdioma(). El método privado Notificar() itera la lista de observers y llama a ActualizarIdioma() en cada uno. El helper T(espagnol, ingles) retorna el texto correcto según el idioma actual.
- IObservadorIdioma (interfaz Observer): contiene el método void ActualizarIdioma(). Cualquier formulario que quiera reaccionar al cambio de idioma debe implementar esta interfaz.
- Form1, FormABM y FormComposite (Observers concretos): implementan IObservadorIdioma y se suscriben a GestorIdioma en su constructor. Se desuscriben en OnFormClosed() para evitar referencias colgantes.

Flujo de cambio de idioma

1. El usuario hace clic en el botón English/Español en la esquina superior derecha de Form1.
2. El evento Click llama a GestorIdioma.Instancia.CambiarIdioma(), que alterna el valor de IdiomaActual.
3. GestorIdioma llama a Notificar(), que itera la lista de observers suscritos.
4. Cada formulario suscrito ejecuta su método ActualizarIdioma(), que actualiza todos los textos de sus controles usando gestor.T("texto ES", "text EN").
5. La Bitácora registra el cambio (idioma anterior → idioma actual).

Alcance de la traducción

Todos los textos de la interfaz están traducidos: títulos de formularios, etiquetas (labels), textos de botones, encabezados de columnas de la grilla, textos de mensajes de confirmación y error, y encabezados de las listas en FormComposite. La traducción se aplica en tiempo real sin necesidad de cerrar ni reabrir los formularios.

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 2/6/2026
	Alumno: Oven Pereyra			Legajo:	
	Localización:	Comisión: 3A	Turno:	Año: 2026	
	Castelar		Noche		Versión
	<Nombre del Sistema>				
2 Entrega					

Diagrama de clase

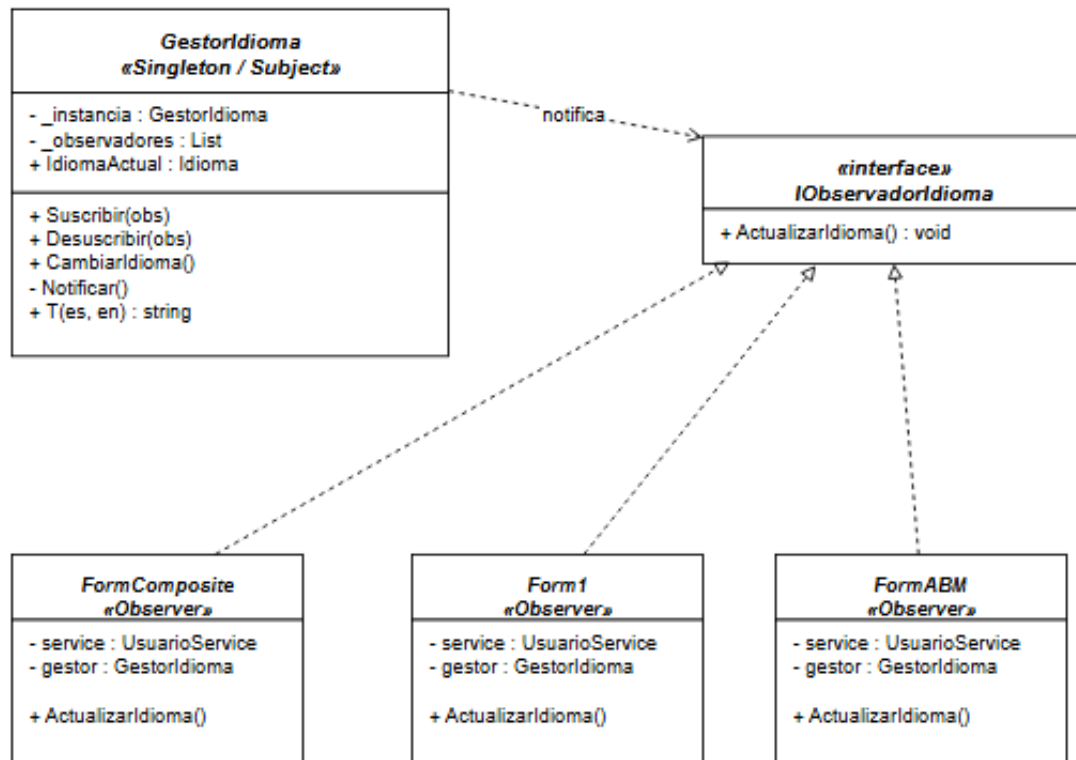
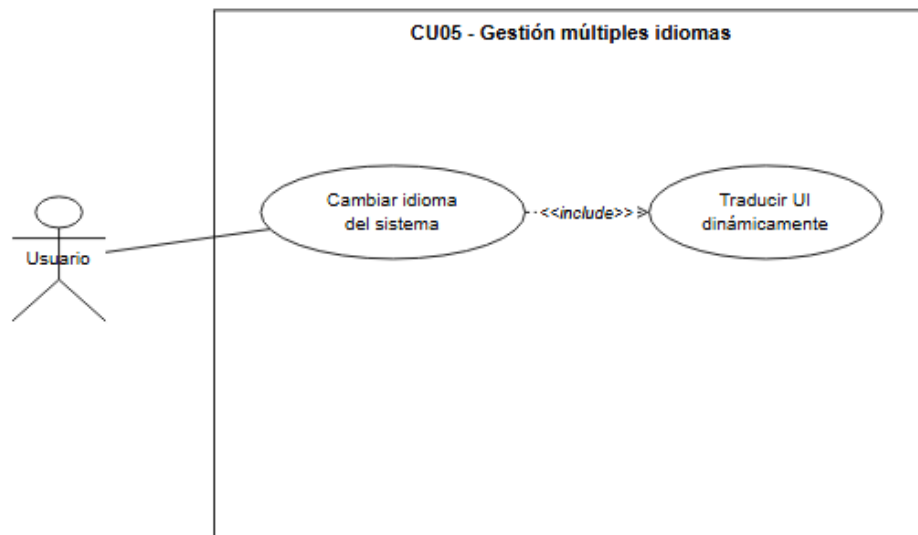


Diagrama de caso de uso

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oven Pereyra		Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026
	<Nombre del Sistema> 2 Entrega			Versión



ID Y NOMBRE:	CU01- Cambiar idioma del sistema
ESTADO:	Aprobado
ACTOR PRINCIPAL: Usuario	
ACTORES SECUNDARIO: -	
PRECONDICIONES: Deben existir idiomas registrados en la base de datos con sus respectivas traducciones	
PUNTOS DE EXTENSIÓN: N/A	
CONDICIÓN: -	
ESCENARIO PRINCIPAL: 1. El usuario despliega la lista de idiomas disponibles 2. El sistema muestra los idiomas obtenidos de la base de datos 3. El usuarios selecciona un idioma diferente al actual 4. El sistema registra el nuevo idioma seleccionado	

UNIVERSIDAD ABIERTA INTERAMERICANA						
Facultad de Tecnología Informática						
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 2/6/2026	
	Alumno: Oven Pereyra			Legajo:		
	Localización:	Comisión: 3A	Turno:	Año: 2026		
	Castelar		Noche			
	<Nombre del Sistema>				Versión	
2 Entrega						

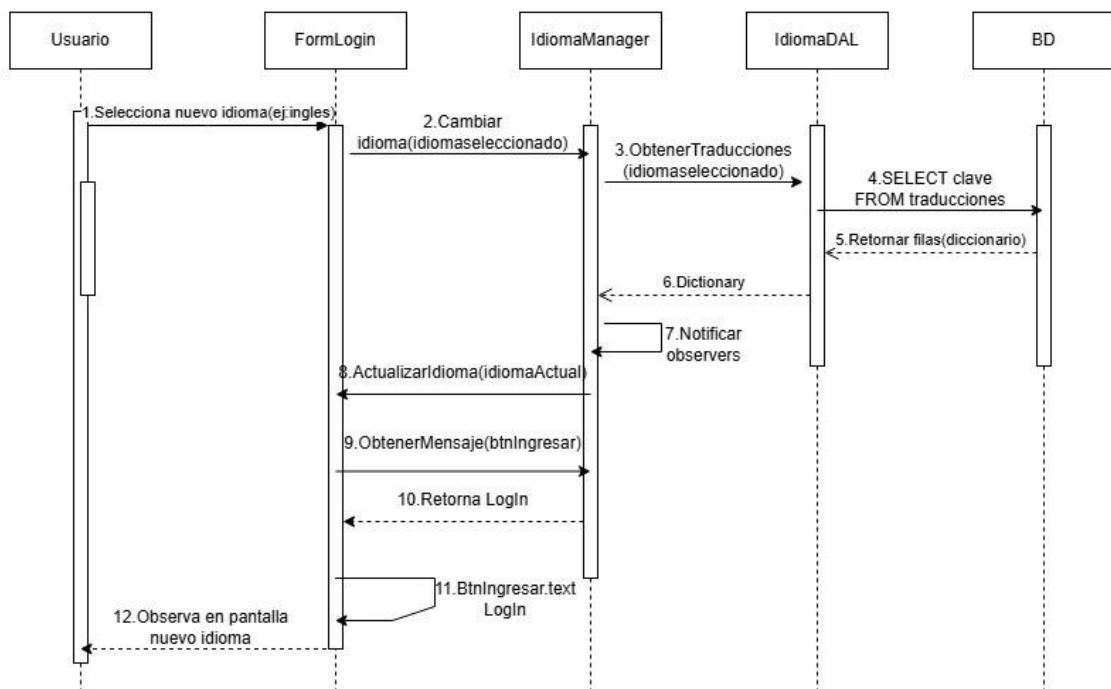
5. El sistema consulta en la base de datos para traer la nueva traducción
6. El sistema notifica dinámicamente a todas las pantallas activas
7. La interfaz actualiza todos sus textos con las traducciones

FLUJOS ALTERNATIVO: -

POSTCONDICIONES: Todos los textos en los formularios abiertos estaran en el nuevo idioma elegido.

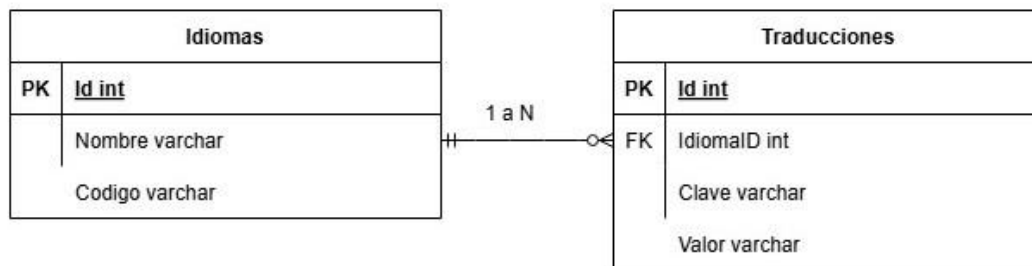
UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 2/6/2026
	Alumno: Oven Pereyra			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	2 Entrega				

Diagrama de secuencia



UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 2/6/2026
	Alumno: Oten Pereyra			Legajo:	
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026	
	<Nombre del Sistema>				Versión
	2 Entrega				

DER



T06. Gestión de Bitácora y Control de cambios

Objetivo

Implementar un mecanismo de auditoría persistente que registre de forma cronológica todas las acciones relevantes realizadas durante la ejecución del sistema. La Bitácora genera un archivo de log por cada ejecución, garantizando trazabilidad completa de los eventos de autenticación, operaciones ABM, cambios de permisos, cambios de idioma y errores de base de datos, sin interrumpir el funcionamiento del sistema en ningún caso.

Descripción detallada

Patrón de diseño: Singleton

La clase Bitacora implementa el patrón Singleton, garantizando que exista una única instancia del logger durante toda la ejecución. El constructor es privado y la instancia se crea de manera lazy cuando se accede por primera vez. Esto asegura que todos los módulos del sistema (UI, BLL) escriban en el mismo archivo de log, sin duplicaciones ni archivos paralelos.

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oven Pereyra			Legajo:
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026
	<Nombre del Sistema> 2 Entrega			Versión

La instancia se inicializa en Program.cs como primera acción al arrancar, antes de que se abra ningún formulario:

```
Bitacora log = Bitacora.Instancia;
```

Generación de archivos de log

Al instanciarse, Bitácora crea automáticamente la carpeta logs/ junto al ejecutable (bin/Debug/logs/) si no existe, y genera un archivo nuevo con el nombre:

```
log_YYYY-MM-DD_HH-mm-ss.log
```

Un archivo nuevo por cada ejecución permite aislar sesiones, facilitando la auditoría y el diagnóstico de problemas. Cada archivo comienza con un encabezado y termina con un pie de cierre, ambos con timestamp.

Thread-safety (lock)

El método privado EscribirLinea() utiliza un objeto de bloqueo estático (_lock) para serializar el acceso al archivo. Esto garantiza que si en el futuro el sistema implementara operaciones multihilo, las entradas del log nunca se corrompan por escrituras concurrentes. Si la escritura falla por cualquier motivo, el bloque catch está vacío intencionalmente: la Bitácora nunca debe interrumpir el flujo principal.

Formato de cada línea

Cada registro tiene el formato fijo:

```
[HH:mm:ss] [NIVEL] [CATEGORIA ] mensaje
```

El nivel se rellena con espacios a la derecha (PadRight(5)) y la categoría a 12 caracteres, lo que genera un alineado visual consistente en cualquier editor de texto.

Niveles y categorías

Nivel	Categoría	Descripción
INFO	INICIO	Inicialización correcta de la base de datos al arrancar
INFO	LOGIN	Usuario inició sesión correctamente
WARN	LOGIN	Intento de login fallido (credenciales incorrectas)

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 2/6/2026
	Alumno: Oven Pereyra			Legajo:	
	Localización:	Comisión: 3A	Turno:	Año: 2026	
	Castelar		Noche		Versión
	<Nombre del Sistema>				
2 Entrega					

INFO	LOGOUT	Usuario cerró sesión
INFO	ABM	Alta, baja o modificación de usuario ejecutada por admin
WARN	ABM	Operación ABM intentada pero fallida (con detalle del error)
INFO	PERMISOS	Permisos guardados para un usuario (tipo y operador)
INFO	IDIOMA	Cambio de idioma (anterior → actual)
ERROR	BD	Error en operación contra SQL Server (con mensaje de excepción)

Eventos registrados

La clase expone métodos con nombre semántico para cada tipo de evento, en lugar de obligar a cada módulo a construir el mensaje manualmente:

- LoginExitoso(string usuario) → INFO/LOGIN
- LoginFallido(string usuario) → WARN/LOGIN
- Logout(string usuario) → INFO/LOGOUT
- UsuarioAgregado(string operador, string nuevo, string rol) → INFO/ABM
- UsuarioModificado(string operador, string usuario) → INFO/ABM
- UsuarioEliminado(string operador, string usuario) → INFO/ABM
- PermisosGuardados(string operador, string destino, string tipo) → INFO/PERMISOS
- Cambioldioma(string anterior, string actual) → INFO/IDIOMA
- ErrorBaseDatos(string operacion, string detalle) → ERROR/BD
- CierreApp() → escribe el pie del log con el timestamp de fin de sesión

Ejemplo de archivo de log

```
=====
BITACORA - Sistema de Usuarios
```

UNIVERSIDAD ABIERTA INTERAMERICANA Facultad de Tecnología Informática				
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo	
	Alumno: Oten Pereyra			Fecha 2/6/2026
	Localización: Castelar	Comisión: 3A	Turno: Noche	Legajo: Año: 2026
	<Nombre del Sistema> 2 Entrega			Versión

Inicio de ejecucion: 10/06/2026 20:45:12

```
=====

[20:45:12] [INFO ] [INICIO      ] Base de datos inicializada correctamente.
[20:45:20] [INFO ] [LOGIN       ] Usuario 'admin' inicio sesion correctamente.
[20:45:35] [INFO ] [ABM         ] 'admin' agrego el usuario 'juan' con rol
'usuario'.
[20:45:50] [WARN ] [LOGIN       ] Intento de login fallido para el usuario
'root'.
[20:46:02] [INFO ] [IDIOMA      ] Idioma cambiado de 'Espanol' a 'Ingles'.
[20:46:18] [INFO ] [PERMISOS    ] 'admin' guardo permisos para 'juan' (tipo:
Administrador).
[20:46:30] [INFO ] [ABM         ] 'admin' modifico el usuario 'juan'.
[20:46:45] [INFO ] [LOGOUT      ] Usuario 'admin' cerro sesion.
```

Fin de ejecucion: 10/06/2026 20:46:50

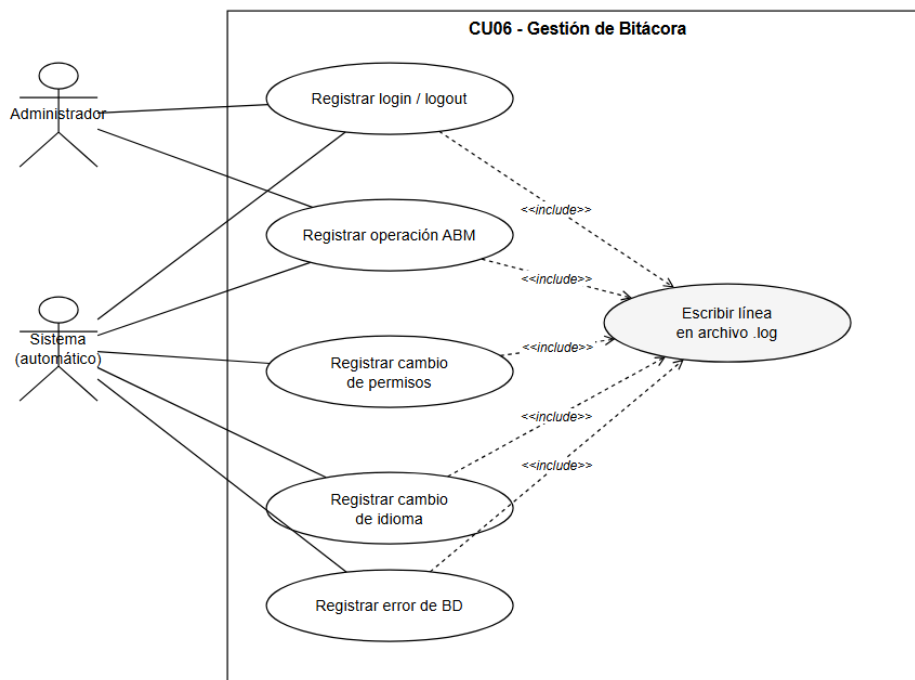
Integración con el resto del sistema

La Bitácora es invocada desde tres puntos del sistema:

- Program.cs: al inicializar la BD (Info/INICIO) y al cerrar la app vía Application.ApplicationExit (CierreApp).
- Form1: LoginExitoso, LoginFallido, Logout y Cambioldioma son registrados desde los event handlers de los botones.
- FormABM: UsuarioAgregado, UsuarioModificado, UsuarioEliminado y la advertencia de operación fallida son registrados tras cada operación ABM.
- FormComposite: PermisosGuardados se registra al confirmar el guardado del árbol de permisos.

UNIVERSIDAD ABIERTA INTERAMERICANA					
Facultad de Tecnología Informática					
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 2/6/2026
	Alumno: Oven Pereyra			Legajo:	
	Localización:	Comisión: 3A	Turno:	Año: 2026	
	Castelar		Noche		Versión
	<Nombre del Sistema>				
2 Entrega					

Diagrama de caso de uso



ID Y NOMBRE:	<u>CU06 - Registrar evento en Bitácora</u>
ESTADO:	Aprobado
ACTOR PRINCIPAL: Sistema	
ACTORES SECUNDARIO: -	
PRECONDICIONES: La aplicación debe estar en ejecución. La carpeta logs/ debe poder crearse o ya existir en bin/Debug/.	
PUNTOS DE EXTENSIÓN: N/A	

UNIVERSIDAD ABIERTA INTERAMERICANA						
Facultad de Tecnología Informática						
	Materia: Ingeniería de software		Docente: Julián Rodríguez Escobedo		Fecha 2/6/2026	
	Alumno: Oven Pereyra			Legajo:		
	Localización: Castelar	Comisión: 3A	Turno: Noche	Año: 2026		
	<Nombre del Sistema>					Versión
	2 Entrega					

CONDICIÓN: - El sistema detecta un evento registrable (login, ABM, cambio de permisos, cambio de idioma, error de BD, cierre de app).

ESCENARIO PRINCIPAL:

1. Un módulo del sistema (Form1, FormABM o FormComposite) invoca el método correspondiente de Bitacora.Instancia
2. Bitácora formatea la línea con timestamp, nivel (INFO/WARN/ERROR), categoría y mensaje
3. EscribirLinea() adquiere el lock y escribe la línea en el archivo .log actual con File.AppendAllText
4. El lock se libera y el flujo principal continúa sin demora

FLUJOS ALTERNATIVO: -

La línea del evento queda escrita en el archivo log_YYYY-MM-DD_HH-mm-ss.log de la ejecución actual.